

rekurrens

Generel idé

Når du analyserer en rekurrensligning, kan du stille dig selv disse spørgsmål:

1. Hvor mange rekursive kald laves der?
2. Hvor stort er problemet i hvert kald?
3. Hvor meget arbejde laves der pr. kald eller pr. niveau?
4. Hvor mange niveauer gentages processen?

Tommelfingerregler

- $T(n - 1) \rightarrow$ lineær dybde
- $T(n / 2) \rightarrow$ logaritmisk dybde
- $2T(n / 2) \rightarrow$ træstruktur
- $+ c \rightarrow$ konstant arbejde
- $+ n \rightarrow$ lineært arbejde

1. $T(n) = T(n - 1) + c$

Intuition

Problemet reduceres med 1 for hvert rekursivt kald, og der udføres konstant arbejde.

Overordnet gennemgang

- Ét rekursivt kald pr. skridt
- Antal skridt:
 $n \rightarrow n-1 \rightarrow n-2 \rightarrow \dots \rightarrow 1 \rightarrow 0$
(n skridt)
- Hvert skridt koster konstant tid

Konklusion

$$T(n)=O(n)T(n) = O(n)T(n)=O(n)$$

$$2. T(n) = T(n - 1) + n$$

Intuition

Som før reduceres problemet med 1, men arbejdet vokser for hvert skridt.

Overordnet gennemgang

- Stadig n rekursive kald
- Arbejde pr. kald: $n + (n-1) + (n-2) + \dots + 1$
- Summen svarer til: $\frac{n(n+1)}{2}$

Konklusion

$$T(n) = O(n^2)$$

$$3. T(n) = T(n / 2) + c$$

Intuition

Problemet halveres hver gang, og der laves konstant arbejde.

Overordnet gennemgang

- Antal niveauer:
Hvor mange gange kan n halveres, før det bliver 1?
 $\rightarrow \log(n)$
- Konstant arbejde pr. niveau

Konklusion

$$T(n) = O(\log n)$$

$$4. T(n) = T(n / 2) + n$$

Intuition

Problemet halveres, men der udføres lineært arbejde på hvert niveau.

Overordnet gennemgang

- Niveau 1: arbejde = n
- Niveau 2: arbejde = $n/2$
- Niveau 3: arbejde = $n/4$
- ...
- Summen bliver: $n + n/2 + n/4 + \dots \leq 2n$

Konklusion

$$T(n) = O(n) \quad T(n) = O(n) \quad T(n) = O(n)$$

Note: Denne rekurrens er *ikke* $O(n \log n)$, da arbejdet falder for hvert niveau.

5. $T(n) = 2T(n/2) + n$

Intuition

Problemet opdeles i to lige store delproblemer, og der udføres lineært arbejde ved sammensmeltning.

Overordnet gennemgang

- Niveau 0: 1 problem $\rightarrow$ arbejde = n
- Niveau 1: 2 problemer $\rightarrow$ samlet arbejde = n
- Niveau 2: 4 problemer $\rightarrow$ samlet arbejde = n
- ...
- Antal niveauer: $\log n$

Konklusion

$$T(n) = O(n \log n) \quad T(n) = O(n \log n) \quad T(n) = O(n \log n)$$

Eksempel: Mergesort

6. $T(n) = 2T(n/2) + 1$

Intuition

Problemet deles i to, men arbejdet pr. kald er konstant.

Overordnet gennemgang

- Rekursionen danner et binært træ
- Antal knuder i træet er proportional med n
- Konstant arbejde pr. knude

Konklusion

$$T(n) = O(n) \quad T(n) = O(n) \quad T(n) = O(n)$$

Samlet oversigt

Rekurrens	Tanke	Big-O
$T(n) = T(n-1) + c$	gør ét ekstra arbejde pr. skridt	$O(n)$
$T(n) = T(n-1) + n$	$1+2+\dots+n$	$O(n^2)$
$T(n) = T(n/2) + c$	halvering	$O(\log n)$
$T(n) = T(n/2) + n$	halvering + fuldt scan hver gang	$O(n)$
$T(n) = 2T(n/2) + n$	mergesort	$O(n \log n)$
$T(n) = 2T(n/2) + 1$	binært træ	$O(n)$

Rekurrens	Big-O
$T(n) = T(n-1) + c$	$O(n)$
$T(n) = T(n-1) + n$	$O(n^2)$
$T(n) = T(n/2) + c$	$O(\log n)$
$T(n) = T(n/2) + n$	$O(n)$
$T(n) = 2T(n/2) + n$	$O(n \log n)$
$T(n) = 2T(n/2) + 1$	$O(n)$

$$\frac{T(N)}{N} = \frac{T(N/2)}{N/2} + 1$$

$$\frac{T(16)}{16} = \frac{T(8)}{8} + 1$$

$$\frac{T(8)}{8} = \frac{T(4)}{4} + 1$$

$$\frac{T(4)}{4} = \frac{T(2)}{2} + 1$$

$$\frac{T(2)}{2} = \frac{T(1)}{1} + 1$$

$$\frac{T(16)}{16} + \frac{T(8)}{8} + \frac{T(4)}{4} + \frac{T(2)}{2} = \frac{T(8)}{8} + \frac{T(4)}{4} + \frac{T(2)}{2} + \frac{T(1)}{1} + 1 + 1 + 1 + 1$$

$$\frac{T(16)}{16} = \frac{T(1)}{1} + 4$$

$$T(16) = T(1)16 + 4 \times 16 \Rightarrow T(N) = N + N \cdot \log N \Rightarrow O(N \cdot \log N)$$

$$T(N) = 2 \sum_{i=0}^{N-1} T(i) + cN$$

$$N \cdot T(N) = 2 \sum_{i=0}^{N-1} T(i) + cN^2$$

$$(N-1) T(N-1) = 2 \sum_{i=0}^{N-2} T(i) + c(N-1)^2$$

$$N T(N) - (N-1) T(N-1) = 2 T(N-1) + cN^2 - c(N-1)^2$$

$$N T(N) = (N+1) T(N-1) + 2cN$$

$$\frac{T(N)}{N+1} = \frac{T(N-1)}{N} + \frac{2c}{N+1}$$

$$\frac{T(N-1)}{N} = \frac{T(N-2)}{N-1} + \frac{2c}{N}$$

$$\frac{T(N-2)}{N-1} = \frac{T(N-3)}{N-2} + \frac{2c}{N-1}$$

$$\frac{T(2)}{3} = \frac{T(1)}{2} + \frac{2c}{3}$$

$$\frac{T(N)}{N+1} = \frac{T(1)}{2} + 2c \sum_{i=2}^N \frac{1}{i}$$

$$T(N) = N + N \log N \Rightarrow T(N) = O(N \log N)$$

Hvad er rekurrens?

- En rekurrens (recurrence) er en ligning, der beskriver en funktion ud fra dens værdi på mindre input.
- Eksempel: $T(N)=2T(N/2)+NT(N)=2T(N/2)+N$
 - den tid det tager at sortere med mergesort er den de to sider ($2T(N/2)2T(N/2)$) på NN som er den sidste gennemgang af NN
- Bruges især til at analysere rekursive algoritmer.
- Viser, hvordan problemet nedbrydes i delproblemer.

Eksempel på rekurrens

- Eksempel fra en algoritme, der opdeler problemet i to halvdele og samler resultaterne på lineær tid:
- $T(N)=2T(N/2)+NT(N)=2T(N/2)+N$
- To rekursive kald på størrelse $N/2N/2$
- Plus lineært arbejde for at kombinere resultaterne.

Hvorfor er rekurrenser vigtige?

- Bruges til at forudsige køretiden for rekursive algoritmer.
- Hjælper med at forstå væksten i beregningstid, når input vokser.
- Typiske anvendelser:
 - Merge Sort: $T(N)=2T(N/2)+O(N)T(N)=2T(N/2)+O(N)$
 - Binær søgning: $T(N)=T(N/2)+O(1)T(N)=T(N/2)+O(1)$
 - * Binær søgning er $O(\log N)O(\log N)$
 - Tower of Hanoi: $T(N)=2T(N-1)+O(1)T(N)=2T(N-1)+O(1)$

Metoder til at løse rekurrenser

1. Substitutionsmetoden – gæt løsningen og bevis den med induktion.
2. *Rekursionstræ-metoden – visualiser arbejdet på hvert niveau og summer det.
 - bryd ned itl base-case og summer op
3. Master-teoremet – direkte formel til mange divide and conquer-algoritmer.

Intuitionen bag rekurrens

- En rekurrens beskriver selv-lighed i beregningen.
- Hvert kald deler problemet i mindre versioner af sig selv.
- Når basis-tilfældet nås, stopper rekursionen.
- Summen af arbejdet i alle niveauer giver den samlede kompleksitet.

Teleskop-kikkert

Vi kan trække den ud og vi kan folde den sammen.

Telescoping

- På dansk teleskopering
- “En teleskopende sum er en sum, hvor de fleste led går ud med hinanden.”
- Eller: “at forkorte en sum ved, at led går ud med hinanden”

Telescoping som bevismetode

- Først opstiller vi rekurrens-ligningen.
- Fx: $T(N) = 2T(N/2) + NT(N) = 2T(N/2) + N$ (merge-sort)
- Herefter opstiller vi en ny ligning, hvor N erstattes med $N/2$.
 - antager N er en potens af 2, fordi det er mergesort
- Hvis vi allerførst dividerer med N på begge sider, når vi frem til ligninger, hvor første udtryk på højresiden er lig venstresiden i næste ligning.
- Når vi kommer til bunds, dvs. til base-case, lægger vi alle venstresider og alle højresider sammen.
- Herefter går ovennævnte udtryk ud med hinanden.
- Til sidst har vi derfor kun venstresiden af første ligning tilbage, og den er lig med højresiden af sidste lignings første udtryk plus summen af højresidernes sidste udtryk.
- Tidskompleksiteten kan nu beregnes ved at gange igennem med N .

Analyse af MergeSort Bliver ved med at substituere NN med $N/2N/2$ indtil der står 1 et eller andet sted.

- $T(N) = 2T(N/2) + NT(N) = 2T(N/2) + N$
- $T(N)/N = T(N/2)/(N/2) + 1$
- $T(N/2)/(N/2) = T(N/4)/(N/4) + 1$
- $T(N/4)/(N/4) = T(N/8)/(N/8) + 1$
- fortsæt til base-case.
- $T(2)/2 = T(1)/1 + 1$
- læg højre- og venstresider sammen og reducer.
- $T(N)/N = T(1)/1 + \log N$
 - halver problem ned til 1 og det er $\log N$:))
 - * alle 1 taller bliver til $\log N$ pga ovenstående.
 - $T(N)/NT(N)/N$ og $T(1)/1 + \log N$ er det eneste der ikke bliver reduceret væk.
- $T(N) = N + N \log N \rightarrow O(N \log N)$

Analyse af worst case QuickSort

- $T(N) = T(N-1) + cN$
- $T(N-1) = T(N-2) + c(N-1)$
- $T(N-2) = T(N-3) + c(N-2)$
- $T(N-3) = T(N-4) + c(N-3)$
- fortsæt til base case
- $T(2) = T(1) + c(2)$

- læg højre- og venstresider sammen og reducér.
- $T(N) = T(1) + c \cdot i = 2N \rightarrow \Theta(N^2)$ $T(N) = T(1) + c \cdot i = 2N \rightarrow \Theta(N^2)$

QuickSelect rekursans ligning starten er glemt

$$T(N) = \frac{1}{N} \left[\sum_{j=0}^{N-1} T(j) \right] + CN$$

Aner ikke hvor C bliver af, men den kunne have ikke lide.

$$\downarrow$$

$$NT(N) = \left[\sum_{j=0}^{N-1} T(j) \right] + N^2$$

$$\downarrow$$

$$(N-1)T(N-1) = \left[\sum_{j=0}^{N-1} T(j) \right] + (N-1)^2$$

Vil gerne af med summationstegnene, så man trækker begge sider fra hinanden eller noget.

$$NT(N) - (N-1)T(N-1) = T(N-1) + 2N$$

$$\downarrow$$

$$NT(N) = NT(N-1) + 2N$$

$$\downarrow$$

$$T(N) = T(N-1) + 2$$

Brug Teleskopning

fold ud:

$$T(N-1) = T(N-2) + 2$$

$$T(N-2) = T(N-3) + 2$$

$$T(N-3) = T(N-4) + 2$$

.....

$$T(2) = T(1) + 2$$

Fold ind igen:

$$T(N) = T(1) + 2N \Rightarrow T(N) = O(N)$$